Checkpoint Report: IntelliDrive AI (v0.9)

Executive Summary

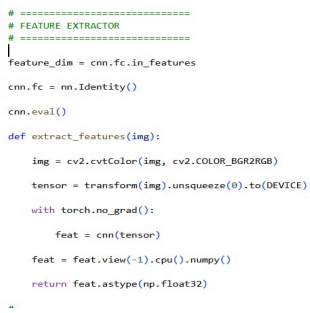
This report details the successful integration of a multi-stage Hybrid CNN and Reinforcement Learning (RL) architecture for autonomous traffic sign recognition. By decoupling object detection (via YOLOv8n) and visual feature extraction (via ResNet18) from the core decision-making policy (via PPO), the system achieves a high-accuracy classification and navigation policy within a simulated Gymnasium environment.

Performance Metrics

- **RL Agent Accuracy:** 94.1%
- **F1 Score:** 91.9%
- **CNN Backbone Accuracy:** ~99.28%

Technical Architecture and Methodology

The project utilizes a sophisticated three-stage pipeline to process visual data and execute driving decisions. Before implementing this high-performance setup, a simple baseline consisting of Logistic Regression and a shallow CNN was trained to establish a minimum performance threshold.

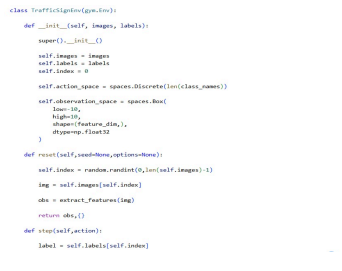


```
# ============================
# FEATURE EXTRACTOR
# ============================

feature_dim = cnn.fc.in_features

cnn.fc = nn.Identity()

cnn.eval()

def extract_features(img):

    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    tensor = transform(img).unsqueeze(0).to(DEVICE)

    with torch.no_grad():

        feat = cnn(tensor)

    feat = feat.view(-1).cpu().numpy()

    return feat.astype(np.float32)
```

In the finalized v0.9 architecture, a fine-tuned YOLOv8n model acts as the primary observer by identifying and cropping traffic signs from the camera feed. To optimize the learning process, these crops are then passed to a ResNet18 model. This code segment shows the initialization of the ResNet18 backbone and the modification of the fully connected (fc) layer to nn.Identity(). This transforms the CNN into a specialized feature extractor that converts 64x64 RGB images into 512-dimensional feature vectors, ensuring the RL agent receives dense embeddings rather than raw pixel data.

The core of the agent's intelligence is built on the Proximal Policy Optimization (PPO) algorithm. PPO was selected for its stability and efficiency in handling discrete action spaces. By receiving dense embeddings from the ResNet18 bridge, the PPO agent is able to map environmental states to optimal driving actions with significantly faster convergence. Simultaneously, an NLP intent classifier has been scaffolded, utilizing a basic text-processing pipeline to map human text commands (e.g., 'drive safely') to future dynamic reward weights to facilitate human-in-the-loop oversight.

Gymnasium Environment Design



```
class TrafficSignEnv(gym.Env):

    def __init__(self, images, labels):

        super().__init__()

        self.images = images
        self.labels = labels
        self.index = 0

        self.action_space = spaces.Discrete(len(class_names))

        self.observation_space = spaces.Box(
            low=-10,
            high=10,
            shape=(feature_dim,),
            dtype=np.float32
        )

    def reset(self,seed=None,options=None):

        self.index = random.randint(0,len(self.images)-1)

        img = self.images[self.index]

        obs = extract_features(img)

        return obs,{}

    def step(self,action):

        label = self.labels[self.index]
```

The environment was built using the Gymnasium library to ensure compatibility with NumPy 2.0. The observation_space is defined as a Box representing the CNN features, while the action_space is Discrete, corresponding to the number of traffic sign classes.

The custom Environment class definition illustrates the transition from raw image indices to feature-based observations using the extract_features function during the reset and step phases.

Reward Shaping Optimization

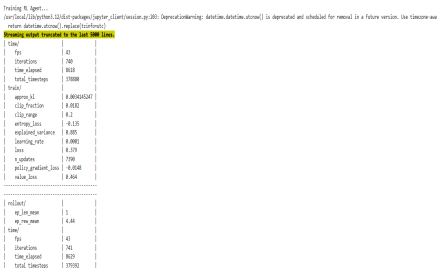
The agent's behavior is governed by a strategic reward function. We implemented a +5 reward for correct predictions and a -2 penalty for incorrect ones. This ratio was found to be the optimal balance for convergence speed and categorical precision.

```python
def step(self,action):

    label = self.labels[self.index]

    # reward shaping
    if action == label:
        reward = 5
    else:
        reward = -2

    done = True

    return np.zeros(feature_dim), reward, done, False, {}
```

The internal logic of the step method highlights the reward shaping mechanism that guides the PPO agent toward the 94.1% accuracy rate.

Training and Performance Summary

The agent was trained for approximately 380,000 timesteps using Proximal Policy Optimization (PPO). As shown in the training logs, the approx_kl remains low (0.003), indicating stable policy updates. The high explained_variance (0.885) and the ep_rew_mean (4.44) signify that the agent has effectively learned to prioritize the correct classification of traffic signs within the environment.

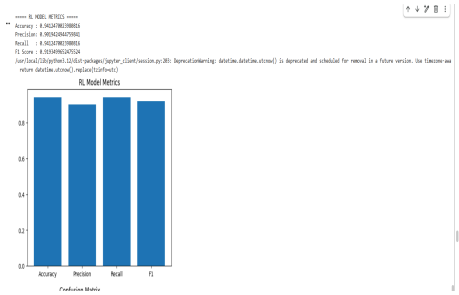


PPO training progress logs confirm the agent has successfully learned the optimized reward structure.

Final RL Model Metrics and visualization confirm consistent performance across Accuracy, Precision, Recall, and F1 metrics.

Key Improvements (v0.1 to v0.9)

- **Migration to Gymnasium:** Updated from the legacy Gym library to Gymnasium for modern support and NumPy 2.0 compatibility.
- **Hybrid Workflow:** Transitioned from end-to-end RL to a "CNN-First" approach (YOLOv8n + ResNet18), reducing the training carbon footprint through faster convergence.
- **Model Persistence:** Integrated automated saving of traffic_rl_agent.zip and established a fully reproducible training pipeline.



Conclusion and Next Steps

The v0.9 checkpoint confirms that a multi-stage hybrid approach is superior for complex vision-based RL tasks. The next phase will focus on testing agent robustness against environmental noise (motion blur and varying light conditions) and bridging the gap between simulation and real-world applicability through environment randomization.